

Reviewer Pack — Monochromatic Rectangle Density Lower Bounds for PARITY Comm...

Entry 604222a146e1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 14:08:42 UTC

Monochromatic Rectangle Density Lower Bounds for PARITY Communication Matrices

Entry ID: 604222a146e1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 14:08:42 UTC

1. Conjecture

Field A (mathematical branch): COMMUNICATION_COMPLEXITY **Field B** (complexity object): AC0_PARITY

Statement:

For any monotone AC^0 formula C computing PARITY on n variables, the number of monochromatic rectangles in its communication matrix is at least $\Omega(n)$, implying that the formula size is exponential in n .

Rationale (proposer's reasoning):

Monochromatic rectangle density in the communication matrix of PARITY captures structural constraints on monotone AC^0 formulas. By linking this combinatorial invariant to the KW game's protocol cost, we derive a lower bound on formula size, bypassing random restriction arguments.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7006db2242bedaf5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=266.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_formula(n):
        # Generate a random monotone AC0 formula for n variables
        formula = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            formula.append(clause)
        return formula

    def compute_communication_matrix(formula, n):
        # Compute the communication matrix for the given formula
        C = [[0] * n for _ in range(n)]
        for clause in formula:
            for x in range(2**n):
                if all((x >> i) & 1 == abs(clause[i]) for i in range(n)):
                    for y in range(2**n):
                        if all((y >> i) & 1 == abs(clause[i]) for i in range(n)):
                            C[x // (2**(n//2))] [y // (2**(n//2))] += 1
        return C

    def count_monochromatic_rectangles(C, n):
        # Count monochromatic rectangles in the communication matrix
        count = 0
        for i in range(n):
            for j in range(i+1, n):
                for k in range(n):
                    for l in range(k+1, n):
                        if (C[i][k] + C[j][l]) == (n // 2) * 4:
                            count += 1
        return count

    n = random.randint(5, 40)
    formula = generate_ac0_formula(n)
    C = compute_communication_matrix(formula, n)
```

```

rectangle_count = count_monochromatic_rectangles(C, n)

metric_name = "monochromatic_rectangle_count"
metric_value = rectangle_count
instances_tested = 1
conjecture_holds = rectangle_count >= n
counterexample = "" if conjecture_holds else f"Formula size {len(formula)}, n={n}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Formula size {len(results[0]['counterexample'])}\", n={n}\")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with extended computation time

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110555
2	preregistration	ollama_remote	qwen3:8b	0	0	24042
3	novelty	ollama_remote	qwen3:8b	0	0	22151
4	novelty	ollama_remote	qwen3:8b	0	0	13929
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14793
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9074
7	judge	ollama_remote	qwen3:8b	0	0	116206

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 310749 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/604222a146e1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/604222a146e1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/604222a146e1.tar.gz` (if generated)